Quorum (New)

Let's learn about Quorum and its usage.

Background

In Distributed Systems, data is replicated across multiple servers for fault tolerance and high availability. Once a system decides to maintain multiple copies of data, another problem arises: how to make sure that all replicas are consistent, i.e., if they all have the latest copy of the data and that all clients see the same view of the data?

Solution

In a distributed environment, a quorum is the minimum number of servers on which a distributed operation needs to be performed successfully before declaring the operation's overall success.

Suppose a database is replicated on five machines. In that case, quorum refers to the minimum number of machines that perform the same action (commit or abort) for a given transaction in order to decide the final operation for that transaction. So, in a set of 5 machines, three machines form the majority quorum, and if they agree, we will commit that operation. Quorum enforces the consistency requirement needed for distributed operations.

In systems with multiple replicas, there is a possibility that the user reads inconsistent data. For example, when there are three replicas, `R1`, `R2`, and `R3` in a cluster, and a user writes value `v1` to replica `R1`. Then another user reads from replica `R2` or `R3` which are still behind `R1` and thus will not have the value `v1`, so the second user will not get the consistent state of data.

What value should we choose for a quorum? More than half of the number of nodes in the cluster: $(N/2+1)$ where N is the total number of nodes in the cluster, for example:

- In a 5-node cluster, three nodes must be online to have a majority.
- In a 4-node cluster, three nodes must be online to have a majority.
- With 5-node, the system can afford two node failures, whereas, with 4-node, it can afford only one node failure. Because of this logic, it is recommended to always have an odd number of total nodes in the cluster.

Quorum is achieved when nodes follow the below protocol: $R+W>N$, where:
N = nodes in the quorum group
W = minimum write nodes
R = minimum read nodes

If a distributed system follows $R+W>N$ rule, then every read will see at least one copy of the latest value written. For example, a common configuration could be (N=3, W=2, R=2) to ensure strong consistency. Here are a couple of other examples:

- (N=3, W=1, R=3): fast write, slow read, not very durable
- (N=3, W=3, R=1): slow write, fast read, durable

The following two things should be kept in mind before deciding read/write quorum:

- R=1 and W=N ⇒ full replication (write-all, read-one): undesirable when servers can be unavailable because writes are not guaranteed to complete.
- Best performance (throughput/availability) when $1<r<w<n$, because reads are more frequent than writes in most applications

How It Works

- **Majority-Based Quorum**: The most common type of quorum where an operation requires a majority (more than half) of the nodes to agree or participate. For instance, in a system with 5 nodes, at least 3 must agree for a decision to be made.
- **Read and Write Quorums**: For read and write operations, different quorum sizes can be defined. For example, a system might require a write quorum of 3 nodes and a read quorum of 2 nodes in a 5-node cluster.

Use Cases

Distributed Databases

- Ensuring consistency in a database cluster, where multiple nodes might hold copies of the same data.

Cluster Management

- In server clusters, a quorum decides which nodes form the 'active' cluster, especially important for avoiding 'split-brain' scenarios where a cluster might be divided into two parts, each believing it is the active cluster.

Consensus Protocols

- In algorithms like Paxos or Raft, a quorum is crucial for achieving consensus among distributed nodes regarding the state of the system or the outcome of an operation.

Advantages

1. **Fault Tolerance**: Allows the system to tolerate a certain number of failures while still operating correctly.
2. **Consistency**: Helps maintain data consistency across distributed nodes.
3. **Availability**: Increases the availability of the system by allowing operations to proceed as long as the quorum condition is met.

Challenges

1. **Network Partitions**: In cases of network failures, forming a quorum might be challenging, impacting system availability.
2. **Performance Overhead**: Achieving a quorum, especially in large clusters, can introduce latency in decision-making processes.
3. **Complexity**: Implementing and managing quorum-based systems can be complex, particularly in dynamic environments with frequent node or network changes.

Conclusion

Quorum is a fundamental concept in distributed systems, playing a crucial role in ensuring consistency, reliability, and availability in environments where multiple nodes work together. While it enhances fault tolerance, it also introduces additional complexity and requires careful design and management to balance consistency, availability, and performance.

← Previous
Bloom Filters (New)

Next →
Leader And Follower (New)

☐ Mark as Completed